

CS541 Project 6: IBL - Image Based Lighting

Synopsis

This project introduces **IBL (Image Based Lighting)** where all the millions of pixels of the skydome image becomes **sources of light**, not sources of background color as in previous projects. An IBL skydome image will be in the “High Dynamic Range” (HDR) format. That is, it with pixels values that are not restricted to the $[0,1]$ range, enabling it to contain sun bright regions of light.

Specific points of an IBL skydome

- It acts as a skydome providing background pixels where geometry is not otherwise visible. (As in a previous project.)
- It provides a hemisphere’s worth of light at any point in the scene. This will **eliminate** the need for an ambient light term.
- It will provide a hemisphere’s worth of light at each point in the scene, all of which will undergo a diffuse reflection according to the diffuse portion of our BRDF calculation.
- For any surface with a specular reflection, i.e., $K_s \neq (0,0,0)$, it will provide a source of light from the specific reflection direction to act as input to the specular reflection portion of our BRDF.



Instructions

Specific changes to the existing project are:

- Read in an HDR skydome instead of whatever (non HDR) skydome you were using. This project comes with a ZIP file of about a dozen HDR sky domes. (See the **Reading HDR images** section for details on reading such images.)
- Read in an IRRADIANCE image derived from the skydome. (Also provided by this project’s ZIP file.)
- Replace all three portions of the lighting calculation (ambient+diffuse+specular) to use the (typically 10-20 million pixels of the) IBL images as the source of light. (See **IBL Lighting Derivation** section below.)
- The single point light we’ve been using all semester will be retired. So too will it’s shadow calculation. Alternatively, keep the point light and shadow calculation, but place the light at a position in the sky to match the image of the sun in the IBL image.
- Modify the very last line of the lighting shader to account for 3 features present in real world lighting situations that have been ignored up to now. (See **Tone Mapping** section below.) The three features are:
 - **Tone Mapping:** mapping lighting calculations from $[0, \infty]$ to a displayable range of $[0,1]$.
 - **Linear Color Space to Gamma Color Space conversion:** Our lighting calculations are in linear color space (so we can add and scale RGB values), but all display screens actually display in gamma color space. A conversion is simple but necessary.
 - **Exposure control:** Simulate an eye’s (or camera’s) ability to operate in a wide range of lighting ranges by adjusting the pupil (or aperture) size.

What to hand in

Be sure your submitted **zip** file contains all **source code**, the **vcxproj** file, all **shader code**, any **textures** you use.

Derivation of the IBL Lighting Integral

In IBL lighting (and real life) we account for light coming from infinitely many directions instead of just a single point light. Every one of the infinitely many sources of input light must go through the BRDF calculation and the infinitely many results summed up. This is an integral, traditionally called the **Rendering Equation**.

$$L_o(\omega_o) = \int_{\Omega} L_i(\omega_i) (N \cdot \omega_i)_+ \text{BRDF}(\omega_i, \omega_o) d\omega_i$$

Notation notes:

- ω_i is the light input direction, replacing L from our previous single point light.
- ω_o is the light output direction, replacing V .
- $L_i(\omega_i)$ is the light coming from the ω_i direction, replacing our I_i .
- $L_o(\omega_o)$ is the light output in the ω_o direction, replacing our I_o .

In this new notation, the BRDF is

$$\text{BRDF}(\omega_i, \omega_o) = \frac{K_d}{\pi} + \frac{D(H) F(\omega_i, H) G(\omega_i, \omega_o, H)}{4 (\omega_i \cdot N) (\omega_o \cdot N)}$$

where $H = (\omega_i + \omega_o) / \|\omega_i + \omega_o\|$ is halfway between ω_i and ω_o .

Substituting the BRDF and separating the integral across its two terms yields

$$L_o(\omega_o) = \int_{\Omega} L_i(\omega_i) (N \cdot \omega_i)_+ \frac{K_d}{\pi} d\omega_i + \int_{\Omega} L_i(\omega_i) (N \cdot \omega_i)_+ \frac{D(H) G(\omega_i, \omega_o, H) F(\omega_i, H)}{4 (\omega_i \cdot N) (\omega_o \cdot N)} d\omega_i \quad (*)$$

These two terms will be dealt with separately in the next two sections.

Image Based Lighting

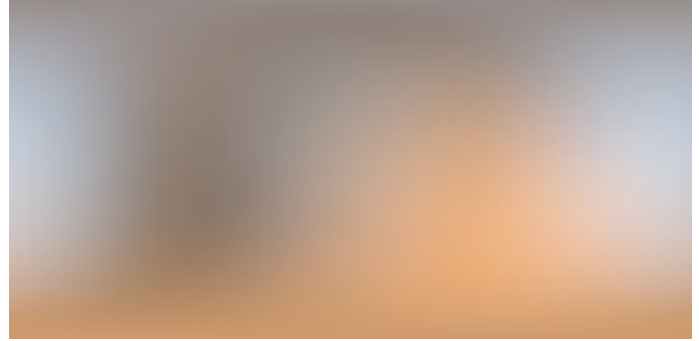
Image Based Lighting (IBL) uses the contents of a skydome image (all 10-20 million pixels) for lighting a scene. IBL also uses a heavily preprocessed version of the skydome image called an **Irradiance Map**.

This project supplies a selection of skydomes and their pre-calculated irradiance maps, both presented as HDR images..

The current lighting calculation which uses a single point light as a source of light will be modified to use all the many pixels of the skydome as sources of light. The lighting calculation will actually simplify from three terms (ambient+diffuse+specular) to two terms (diffuse+specular) with the new diffuse term taking on the rolls of both the original ambient and diffuse portion.



A sample full resolution HDR image



Low resolution irradiance HDR image

Diffuse lighting calculation:

The diffuse portion of (*) simplifies to

$$\int_{\Omega} L_i(\omega_i) (N \cdot \omega_i)_+ \frac{K_d}{\pi} d\omega_i = \frac{K_d}{\pi} \int_{\Omega} L_i(\omega_i) (N \cdot \omega_i)_+ d\omega_i \cdot$$

The integral can be precalculated for any N and stored in an HDR image called the irradiance map.

So in your shader, the full diffuse (and ambient) lighting calculation at a point with normal N becomes:

$$\text{diffuse} = \frac{K_d}{\pi} \text{irradiance}(N)$$

where the irradiance map is read at a uv calculated from N via the usual atan, acos calculation.

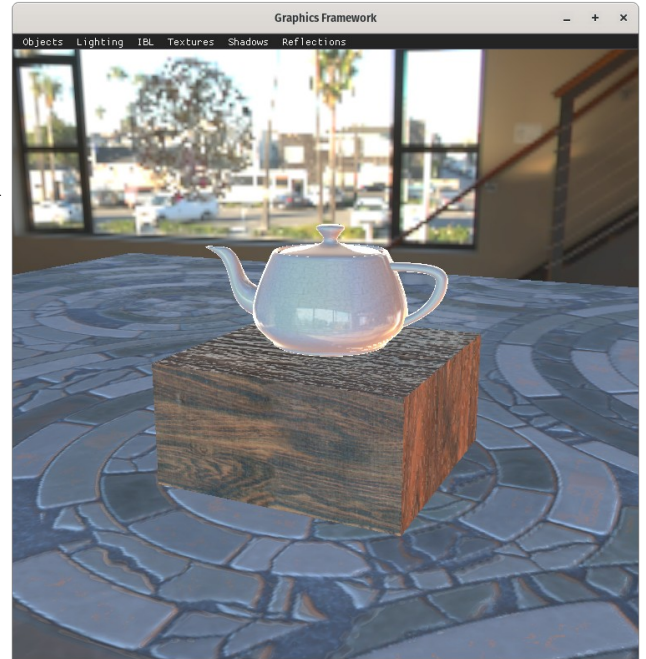
Specular lighting calculation:

For the specular portion of (*) we make the simplifying assumption that the specular reflection is zero everywhere except in one specific direction - that being the reflection direction $R = 2(V \cdot N)N - V$. The integral then reduces to a single calculation of the specular portion of the BRDF with:

- A light input direction of R
- A light value from the skydome pixel in the direction R :

So

$$\text{specular} = \text{skydome}(R) (N \cdot R)_+ \frac{D(H) G(R, V, H) F(R, H)}{4 (R \cdot N) (V \cdot N)}$$



The final image with diffuse lighting coming from the irradiance map, and specular lighting from the high resolution sky-dome HDR image.

Full lighting calculation: Just *diffuse+specular* with no ambient term since the diffuse term accounts for a full hemisphere of light input.

Tone Mapping, Exposure -Control, and Linear Color space

The equation to implement all three features is

$$C_{out} = \left(\frac{e C_{in}}{(e C_{in} + (1,1,1))} \right)^{1/2.2}$$

where C_{in} is the RGB you've previously calculated for output, and C_{out} is the new converted actual RGB output. This should be the last line in fragment shader. The value that was previously assigned to **FragColor** is converted with this equation and its output is assigned to **FragColor**.

The three features are:

- **The tone mapping part:** The $C_{in}/(C_{in}+1)$ portion of this equation converts RGB values from an infinite range $[0 \dots \infty]$ to a displayable range $[0 \dots 1]$. This is a global tone-mapping operator. Complex local-tone-mapping operators also exist but are outside the scope of this project.
- **The exposure control part:** The $e C_{in}$ portion of this equation provides a scale factor e to scale the input RGB values into a comfortable range for viewing. This simulates an eye's (or camera's) ability to adjust size of the pupil (or aperture). You can set it to a constant that works well with whatever IBL image you choose, or you can provide a GUI slider to adjust it. You may have noticed games adjust this value whenever your game character moves between bright and dark locations in the game world.
- **The color space conversion part:** The exponent of $1/2.2$ converts your linearly calculated RGB values to **sRGB** values as expected by all output devices. This exponent cancels out the display's built-in implicit exponent of 2.2 this ensuring our calculated RGB color is what is actually displayed by the device. For a little extra control, you may want to include an extra adjustable contrast parameter k in the exponent as $k/2.2$ instead of $1/2.2$. This contrast parameter is what games provide as "gamma" or "brightness" controls.

Reading HDR images for OpenGL Textures

For reading HDR images into OpenGL textures I suggest you derive a new class, named **HDR**, from the current class **Texture**, and make two modifications in its constructor:

- For reading an **HDR** image, change the **stbi_load** line to:
`float* image = stbi_loadf(path.c_str(), &width, &height, &depth, 4);`
(Note the one letter change from **load** to **loadf**, and the return type of **float***)
- Change the **glTexImage2D** line to account for the image being floats instead of unsigned ints:
`glTexImage2D(GL_TEXTURE_2D, 0, (GLint)GL_RGBA, width, height, 0, GL_RGBA, GL_FLOAT, image);`

That's all! A texture created this way is used just as is one created with the original code.